

Metoda DFS - przeglądanie grafu w głąb

(depth first search)

Kojarzona z metodą nawrotów (backtracking):

- idź do kolejnych nowych wierzchołków najdalej jak się da, jeśli dalej nie można, to wycofaj się do poprzedniego wierzchołka i próbuj inną krawędzią.

Realizacja:

- reprezentacja grafu: dowolna, pozwalająca efektywnie przeglądać listy następników/sąsiadów;
- zatem preferowana reprezentacja za pomocą list sąsiedztwa Adj[u];
- algorytm podobny do BFS, stos zamiast kolejki;
- stos można zrealizować niejawnie, za pomocą rekurencji.

DFS wersja 1 - najprostsza:

Algorytm SimpleDFS(G):

```
for each u ∈ V(G) do
    visited[u] := false;
for each u ∈ V(G)
    if not visited[u] then
        SimpleDFS_(u);
```

end;

procedure SimpleDFSVisit(u)

```
    visited[u] := true;
    PreorderVisit(u);           // badanie "na wejściu"
    for each v ∈ Adj[u] do      // badaj krawędź (u,v)
        if not visited[v] then
            SimpleDFSVisit(v);
    PostorderVisit(u);          // badanie "na wyjściu"
```

end.

DFS wersja 2 - pełna:

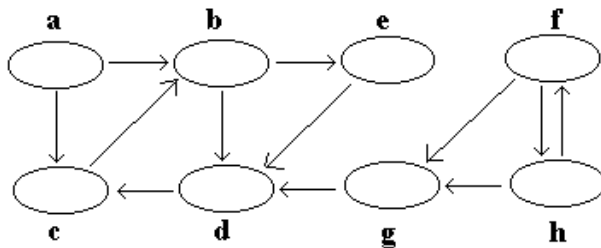
- 3 kolory,
- drzewo rozpinające,
- czas wejścia i wyjścia.

Algorytm DFS(G) :

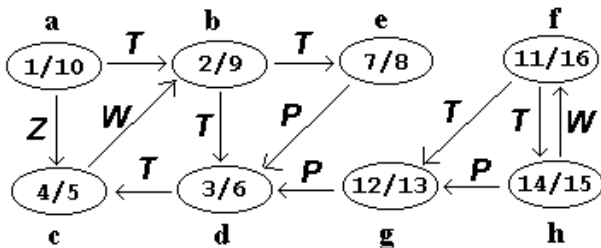
```
for each  $u \in V(G)$  do
     $c[u] := \text{white};$            // kolor
     $p[u] := \text{NULL};$          // poprzednik w drzewie
time := 0;
for each  $u \in V(G)$  do
    if  $c[u] = \text{white}$  then
        DFSVisit(u);
end;
```

procedure DFSVisit(u)

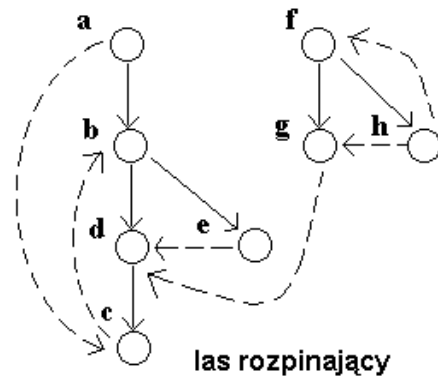
```
     $c[u] := \text{grey};$            // preorder visit
     $d[u] := ++\text{time};$          // czas wejścia - preorder
    for each  $v \in \text{Adj}[u]$  do    // badaj krawędź (u,v)
        if  $c[v] = \text{white}$  then
             $p[v] := u;$ 
            DFSVisit(v);
        else
            zależnie od zastosowań, być może puste;
     $c[u] := \text{black};$            // postorder visit
     $f[u] := ++\text{time};$          // czas wyjścia - postorder
end.
```



w pętli głównej oraz w listach następników
kolejność alfabetyczna



$d(v)/f(v)$ - moment odkrycia/opuszczenia wierzchołka



krawędzie:
T - drzewowa
W - wsteczna
P - poprzeczna
Z - wzdłużna

Złożoność:

- część główna (algorytm DFS): $\Theta(n)$;
- dzięki kolorowaniu, DFSVisit(u) wywoływana dokładnie raz dla każdego wierzchołka u , czy to z części głównej, czy podczas wykonywania DFSVisit dla innych wierzchołków;
- zatem czas wykonania początkowych i końcowych instrukcji DFSVisit: $\Theta(n)$;
- pętla "for each $v \in \text{Adj}(u)$ " ma koszt $\Theta(|\text{Adj}(u)|)$;
- zatem razem wszystkie te pętle mają koszt

$$\sum_{u \in V} \Theta(|\text{Adj}(u)|) = \Theta(m);$$

- (inaczej: każda krawędź grafu badana dokładnie raz).

Zatem DFS wykonuje się w czasie $\Theta(n+m)$.

Własności przeglądu DFS

- podawane bez formalnego dowodu,
- dotyczą zarówno grafów skierowanych jak i nieskierowanych (chyba że zaznaczono inaczej),
- dotyczą dowolnego przeglądu DFS, tzn. przy dowolnej kolejności wywołań w części głównej oraz dowolnie ustalonej kolejności węzłów na każdej liście sąsiedztwa.

Fakt. (Las rozpinający przeszukiwania w głąb)

Graf $T=(V(G), E(T))$, gdzie

$$E(T)=\{e=(p(u),u), u\in V(G)\}$$

jest lasem rozpinającym dla G .

(Przy odpowiednio zdefiniowanym skierowanym drzewie rozpinającym).

Fakt.

Dla grafu nieskierowanego spójnego las przeszukiwania w głąb składa się z jednego drzewa.

Twierdzenie (o nawiasach)

W wyniku przeszukiwania grafu w głąb dla dowolnych dwóch wierzchołków u, v zachodzi dokładnie jeden z trzech poniższych warunków dotyczących przedziałów czasowych między odkryciem węzła a zakończeniem jego przetwarzania:

$$[d(u), f(u)] \cap [d(v), f(v)] = \emptyset, \text{ albo}$$

$$[d(u), f(u)] \subseteq [d(v), f(v)], \text{ albo}$$

$$[d(v), f(v)] \subseteq [d(u), f(u)].$$

Wniosek (o zagnieżdżaniu przedziałów)

Wierzchołek v jest potomkiem wierzchołka u , $u \neq v$, w lesie przeszukiwania w głąb wtedy i tylko wtedy gdy

$$d(u) < d(v) < f(v) < f(u),$$

czyli wtedy i tylko wtedy, gdy w momencie wywołania $\text{DFS}(v)$ wierzchołek u jest szary.

(Wynika bezpośrednio z twierdzenia o nawiasach).

Związek DFS-u z osiągalnością stanowi:

Twierdzenie (o białej ścieżce)

Wierzchołek v jest potomkiem wierzchołka u w lesie przeszukiwania w głąb grafu (skierowanego lub nieskierowanego) wtedy i tylko wtedy, gdy w chwili $d(u)$ istnieje biała ścieżka z u do v (tj. ścieżka samych białych wierzchołków).

Klasyfikacja krawędzi w algorytmie DFS

Graf skierowany:

(u,v) - badana krawędź, u - węzeł aktualny.

1. drzewowa (do następnika w drzewie):

$$c(v)=\text{white},$$

2. powrotna (wsteczna, do przodka w drzewie):

$$c(v)=\text{grey},$$

3. w przód (wzdłużna, do potomka w drzewie):

$$c(v)=\text{black} \text{ oraz } d(u) < d(v),$$

4. poprzeczna (do starszego poddrzewa):

$$c(v)=\text{black} \text{ oraz } d(u) > d(v).$$

Graf nieskierowany:

Założenia:

- każda krawędź badana jest dwukrotnie, bo pojawia się na dwóch listach sąsiedztwa;
- zakładamy, że typ krawędzi ustalany jest podczas pierwszego jej badania.

Fakt:

Podczas przeglądu grafu nieskierowanego DFS-em żadna krawędź nie jest klasyfikowana jako poprzeczna lub w przód.

Czyli: w przeglądzie DFS grafu nieskierowanego mamy tylko krawędzie drzewowe i powrotne.

Zastosowania DFS-u

1. Problem osiągalności

Dla zadanych wierzchołków u , v pytamy czy istnieje ścieżka z u do v .

Rozwiązanie oczywiste, zarówno metodą DFS jak i BFS.

2. Spójne składowe grafu nieskierowanego

Wszystkim wierzchołkom składowej przypisać numer tej składowej (różny dla różnych składowych), czyli:

Obliczyć tablicę liczbową $ss[u]$, przy czym

$$ss[u] = ss[v]$$

wtedy i tylko wtedy gdy u i v są w tej samej składowej.

Rozwiązanie:

- wystarczy SimpleDFSVisit,
- zmienną globalną k, zerowaną na początku, zwiększamy o 1 przed każdym wywołaniem funkcji DFSVisit z głównej części algorytmu DFS;
- na początku funkcji DFSVisit dodajemy instrukcję $ss[u] := k$;

Analogicznie można rozwiązać ten problem metodą BFS.

3. Sprawdzanie czy graf zawiera cykle

Fakt.

- Graf zawiera cykl wtedy i tylko wtedy gdy w dowolnym drzewie DFS istnieje krawędź powrotna;
- Dla dowolnego drzewa (lasu) DFS każdy cykl w grafie zawiera co najmniej jedną krawędź powrotną.

W "trójkolorowym" DFS wewnątrz funkcji DFSVisit(u) rozwijamy część "else".

Wersja dla grafu skierowanego:

```
for each  $v \in Adj[u]$  do      // badaj krawędź (u,v)
    if  $c[v] = white$  then
        DFS_Visit(v);
    else
        if  $c[v] = grey$  then
            CYCLE := true; // globalna flaga
```

Dla grafu nieskierowanego podczas przeglądu listy Adj(u) trzeba dodatkowo wykluczyć wierzchołek p(u) (gdyż w grafie nieskierowanym cykle składają się z co najmniej 3 krawędzi):

```
... else
    if c[v] = grey and v ≠ p[u] then
        CYCLE := true;
```

Wniosek:

Acykliczność grafu można sprawdzić za pomocą DFS w czasie

- $\Theta(n+m)$ – dla grafu skierowanego
- $\Theta(n)$ – dla grafu nieskierowanego (przeгляд można przerwać po znalezieniu pierwszej krawędzi powrotnej, jeśli taka istnieje to trafimy na nią nie później niż po przeglądzie $n-1$ krawędzi.)

4. Sortowanie topologiczne

Dane: Graf skierowany $G=(V,E)$ bez cykli (czyli: DAG)

Wynik: liniowy porządek $v_1, \dots, v_n$ wierzchołków taki, że dla każdej krawędzi $(v_i, v_j) \in E$ zachodzi $i < j$.

Czyli:

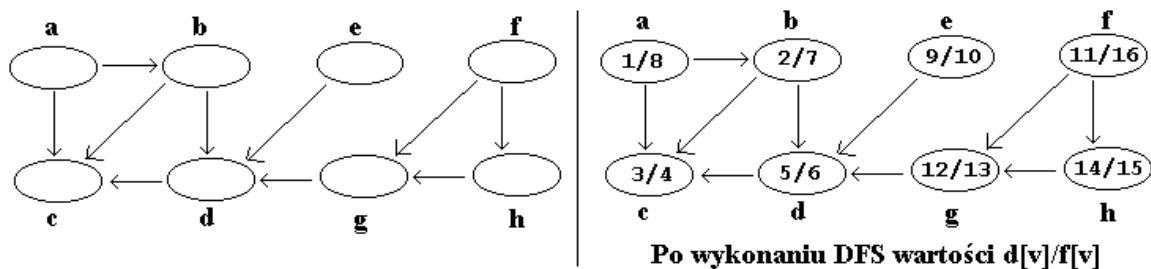
krawędź (u,v) wymusza, aby w ciągu wynikowym wierzchołek u wystąpił wcześniej niż v .

Mnóstwo zastosowań, np.:

- planowanie czynności w procesie produkcyjnym,
- obliczanie najdłuższej ścieżki w sieci acyklicznej.

Rozwiązanie:

- wykonaj DFS obliczający czas postorder $f(u)$, dla każdego wierzchołka u ,
- wypisz wierzchołki w kolejności malejących wartości $f(u)$.



W kolejności malejących $f[v]$ otrzymujemy porządek topologiczny:

f h g e a b d c

Uwaga:

w tej wersji algorytm wymaga sortowania wierzchołków względem wartości $f(u)$, czyli faktycznie odwrócenia permutacji zadanej tablicą $f[]$.

Odwracanie permutacji jest wykonalne w czasie $O(n)$, ale wymaga dodatkowej pamięci wielkości $O(n)$ (wystarczy n bitów).

Wersja ze stosem:

czyli również z dodatkową pamięcią wielkości $O(n)$

- utwórz stos roboczy S początkowo pusty,
- wykonaj DFS, podczas kolorowania na czarno (postorder visit) wstawiaj na stos S ,
- po zakończeniu DFS wypisz zawartość stosu S .

Poprawność:

Założenie: G jest DAG-iem.

Wystarczy wykazać, że jeśli $(u,v) \in E(G)$, to $f(v) < f(u)$.

Gdy (u,v) jest badana to u jest szary, natomiast v

- nie jest szary, wpp. (u,v) zamykałaby cykl;
- jeśli jest biały, to $d(u) < d(v) < f(v) < f(u)$, z tw. o nawiasach;
- jeśli jest czarny, to $f(v) < \text{aktualny czas} < f(u)$ (bo u jeszcze nie rozpatrzone do końca).

QED.

Równie efektywny, a dla niektórych zastosowań bardziej przydatny, jest algorytm sortowania topologicznego metodą eliminacji krawędzi ("obgryzanie grafu") przedstawiony w dalszej części notatek.

5. Znajdowanie cyklu Eulera

Cykl Eulera: cykl w grafie przechodzący wzdłuż każdej krawędzi dokładnie raz.

Twierdzenie (Euler):

Nieskierowany graf $G=(V,E)$ posiada cykl Eulera wtedy i tylko wtedy gdy G jest spójny oraz stopień każdego wierzchołka w G jest parzysty.

Idea algorytmu

(nie jest to bezpośrednio zastosowanie algorytmu DFS)

Wejście: graf spójny $G=(V,E)$.

- zacznij w dowolnym wierzchołku s ;
- przeglądaj graf, idąc w każdym kroku do następnego wierzchołka wzdłuż krawędzi, którą do tej pory nie szedłeś;
- zatrzymujesz się w pewnym wierzchołku u , bo nie ma z niego wyjścia.

Fakt: $u=s$.

Uzasadnienie: parzystość stopni.

Krawędzie którymi przeszliśmy tworzą zatem cykl C_1 .

- wystartuj ponownie z dowolnego wierzchołka, z którego wychodzi nieodwiedzona krawędź;
- powtarzaj przeglądanie jak wyżej aż wrócisz do tego wierzchołka, otrzymując w ten sposób cykl C_2 ;
- powtarzaj znajdowanie cykli aż krawędzie się wyczerpią;
- połącz wszystkie otrzymane cykle w jeden cykl – jest to cykl Eulera.

Pozostają do opracowania szczegóły techniczne: reprezentacja cykli i ich sklejanie tak, aby otrzymać algorytm o złożoności liniowej (ćwiczenie).